

بسمه تعالی

پروژه درس برنامه نویسی پیشرفته

دانشکده فنی انقلاب اسلامی

دانشگاه ملی مهارت

استاد درس: دکتر اردستانی

نام و نام خانوادگی دانشجو: شایان عباسی

شماره دانشجویی: 04121040768022

نکته: در تمام این پروژه ها a آخرین رقم غیر صفر شماره دانشجویی شماست.

پروژه 5: بازی تعادلی پاندول معکوس

شرح

یک پاندول معکوس را شبیه‌سازی کنید.

هدف:

- سقوط نکند.
- کاربر یا کنترلر آن را متعادل نگه دارد.

کلاس‌ها

Pendulum

Cart

Controller

Simulation

GUI

Tkinter Canvas

یا

Pygame

نمره ویژه:

اضافه کردن نویز سنسور

1. کتابخانه ها

```
import tkinter as tk
```

در این خط، کتابخانه `tkinter` را وارد کردم و نام مستعار `tk` به آن دادم. از این کتابخانه برای ساخت پنجره اصلی برنامه، `Canvas` برای رسم پاندول و دکمه‌های کنترل استفاده کردم.

```
import math
```

کتابخانه `math` را برای محاسبات مثلثاتی وارد کردم. از توابع `sin` و `cos` برای محاسبه شتاب زاویه‌ای پاندول و تبدیل موقعیت زاویه‌ای به مختصات `x` و `y` برای رسم روی `Canvas` استفاده می‌شود.

۲. کلاس Pendulum (مدل پاندول)

```
class Pendulum:
```

```
    def __init__(self):
        self.angle = 0.1
        self.angle_vel = 0.0
        self.length = 120
        self.gravity = 9.8
```

این کلاس را برای مدل‌سازی پاندول معکوس طراحی کردم. زاویه اولیه 0.1 رادیان تنظیم شده تا پاندول کمی کج باشد و شبیه‌سازی سریع‌تر آغاز شود. `angle_vel` سرعت زاویه‌ای، `length` طول میله و `gravity` شتاب گرانش را نگه می‌دارند.

متد update (به‌روزرسانی پاندول)

```
def update(self, cart_acc, dt):
    angle_acc = (self.gravity * math.sin(self.angle)
                 - cart_acc * math.cos(self.angle)) / self.length
    self.angle_vel = self.angle_vel + angle_acc * dt
    self.angle_vel = self.angle_vel * 0.995
    self.angle = self.angle + self.angle_vel * dt
```

این متد معادله دیفرانسیل پاندول معکوس را پیاده‌سازی می‌کند. شتاب زاویه‌ای از نیروی گرانش و شتاب چرخ محاسبه می‌شود. ضریب ۰.۹۹۵ میرایی کوچکی اضافه می‌کند تا نوسانات بی‌نهایت نشوند. سپس سرعت و زاویه با روش اویلر به‌روز می‌شوند.

متد is_fallen (تشخیص سقوط)

```
def is_fallen(self):  
    if self.angle > 1.0 or self.angle < -1.0:  
        return True  
    return False
```

این متد بررسی می‌کند آیا پاندول سقوط کرده یا نه. اگر زاویه از حدود ۵۷ درجه بیشتر یا کمتر شود پاندول سقوط کرده تلقی می‌شود و بازی تمام می‌شود.

۳. کلاس Cart (مدل چرخ)

```
class Cart:  
    def __init__(self):  
        self.x = WIDTH / 2  
        self.vel = 0.0  
        self.acc = 0.0  
        self.force = 0.0
```

این کلاس را برای مدل‌سازی چرخ یا ارابه پاندول طراحی کردم. چرخ در وسط صفحه شروع می‌کند. force نیروی اعمال شده توسط کنترلر یا کاربر، acc شتاب، vel سرعت و x موقعیت افقی چرخ را نگه می‌دارند.

متد update (به‌روزرسانی چرخ)

```
def update(self, dt):  
    self.acc = self.force / 5.0  
    self.vel = self.vel + self.acc * dt  
    self.vel = self.vel * 0.9  
    self.x = self.x + self.vel * dt  
    if self.x < 50:  
        self.x = 50
```

```

self.vel = 0.0

if self.x > WIDTH - 50:

    self.x = WIDTH - 50

    self.vel = 0.0

```

این متد حرکت چرخ را شبیه‌سازی می‌کند. شتاب از نیرو تقسیم بر جرم ۵ محاسبه می‌شود. ضریب ۰.۹ اصطکاک را مدل می‌کند تا چرخ به تدریج کند شود. در نهایت چرخ در محدوده ۵۰ تا WIDTH-50 نگه داشته می‌شود تا از صفحه خارج نشود.

۴. کلاس Controller (کنترلر PD)

```

class Controller:

    def __init__(self):

        self.kp = 50.0

        self.kd = 10.0

```

این کلاس را برای کنترل خودکار پاندول طراحی کردم. از یک کنترلر PD ساده استفاده کردم. kp ضریب تناسبی است که به زاویه واکنش نشان می‌دهد و kd ضریب مشتقی است که به سرعت تغییر زاویه واکنش نشان می‌دهد و نوسانات را کاهش می‌دهد.

متد compute (محاسبه نیروی کنترلی)

```

def compute(self, angle, angle_vel):

    force = self.kp * angle + self.kd * angle_vel

    if force > 200:

        force = 200

    if force < -200:

        force = -200

    return force

```

این متد نیروی لازم برای متعادل نگه داشتن پاندول را محاسبه می‌کند. نیروی خروجی ترکیب خطی زاویه و سرعت زاویه‌ای است. خروجی بین -200 و 200 محدود می‌شود تا نیروی غیرواقعی اعمال نشود. هرچه پاندول بیشتر کج شود نیروی بیشتری برای برگرداندن آن اعمال می‌شود.

۵. کلاس Simulation (مدیریت شبیه‌سازی)

class Simulation:

```
def __init__(self):  
    self.pendulum = Pendulum()  
    self.cart = Cart()  
    self.controller = Controller()  
    self.running = False  
    self.dt = 0.03
```

این کلاس همه اجزای شبیه‌سازی را کنار هم قرار می‌دهد. یک Pendulum، یک Cart و یک Controller ساخته می‌شوند. running وضعیت اجرا و dt گام زمانی ۰.۰۳ ثانیه را نگه می‌دارد که برای دقت محاسبات مناسب است.

متد reset (ریست شبیه‌سازی)

```
def reset(self):  
    self.pendulum = Pendulum()  
    self.cart = Cart()  
    self.running = False
```

این متد شبیه‌سازی را به حالت اولیه برمی‌گرداند. اشیاء Pendulum و Cart از نو ساخته می‌شوند تا تمام حالت‌های قبلی پاک شوند و running به False تنظیم می‌شود.

متد step (گام شبیه‌سازی)

```
def step(self):  
    self.cart.update(self.dt)  
    self.pendulum.update(self.cart.acc, self.dt)
```

این متد یک گام زمانی کامل را اجرا می‌کند. ابتدا چرخ به‌روز می‌شود تا شتاب جدید آن محاسبه شود، سپس این شتاب به پاندول داده می‌شود تا معادله دیفرانسیل حل شود. ترتیب اجرا مهم است زیرا پاندول به شتاب به‌روز شده چرخ نیاز دارد.

```
class GUI:
    def __init__(self, root):
        self.root = root
        self.root.title("Inverted Pendulum")
        self.sim = Simulation()
        self.canvas = tk.Canvas(root, width=WIDTH, height=HEIGHT, bg="white")
```

این کلاس را برای مدیریت کامل رابط گرافیکی طراحی کردم. یک `Simulation` ساخته می‌شود و `Canvas` برای رسم پاندول و چرخ تعریف می‌شود. دکمه‌های `Start`، `Stop` و `Reset` و همچنین کلیدهای جهت‌دار برای کنترل دستی به پنجره اضافه می‌شوند.

متدهای `push_left / push_right` (کنترل دستی)

```
def push_left(self, event):
    self.sim.cart.force = -150

def push_right(self, event):
    self.sim.cart.force = 150
```

این دو متد به کلیدهای جهت‌دار چپ و راست متصل هستند. وقتی کاربر کلید فشار می‌دهد نیروی ۱۵۰ واحدی به چرخ اعمال می‌شود. در حلقه `loop` این نیرو با ضریب ۰.۸ کاهش می‌یابد تا یک ضربه لحظه‌ای شبیه‌سازی شود نه نیروی دائمی.

متد `loop` (حلقه اصلی بازی)

```
def loop(self):
    if self.sim.running == True:
        self.sim.step()
        self.sim.cart.force = self.sim.cart.force * 0.8
        if self.sim.pendulum.is_fallen() == True:
            self.sim.running = False
            self.status.config(text="FALLEN! Press Reset", fg="red")
        self.draw()
        self.root.after(30, self.loop)
```

این متد حلقه اصلی بازی است و هر ۳۰ میلی‌ثانیه اجرا می‌شود. یک گام شبیه‌سازی اجرا می‌شود، نیروی دستی کاهش می‌یابد و وضعیت سقوط بررسی می‌شود. اگر پاندول سقوط کند بازی متوقف می‌شود و پیام **FALLEN** نمایش داده می‌شود.

متد draw (رسم صحنه)

```
def draw(self):  
    self.canvas.delete("all")  
    self.canvas.create_line(0, CART_Y+20, WIDTH, CART_Y+20, fill="black", width=2)  
    cx = self.sim.cart.x  
    self.canvas.create_rectangle(cx-40, CART_Y-25, cx+40, CART_Y, fill="steelblue")  
    self.canvas.create_oval(cx-25, CART_Y-7, cx-11, CART_Y+7, fill="gray")  
    self.canvas.create_oval(cx+11, CART_Y-7, cx+25, CART_Y+7, fill="gray")  
    px = cx + length * math.sin(angle)  
    py = CART_Y - 25 - length * math.cos(angle)  
    self.canvas.create_line(cx, CART_Y-25, px, py, fill="black", width=3)  
    self.canvas.create_oval(px-10, py-10, px+10, py+10, fill="orange")
```

این متد هر فریم کل صحنه را از نو رسم می‌کند. ابتدا یک خط افقی به عنوان زمین رسم می‌شود. چرخ آبی رنگ با دو چرخ خاکستری رسم می‌شود. موقعیت سر پاندول با $\sin$ و $\cos$ از زاویه و طول محاسبه می‌شود. میله با یک خط ضخیم و توپ با دایره نارنجی رسم می‌شود.

۷. نقطه ورودی برنامه

```
root = tk.Tk()  
app = GUI(root)  
root.mainloop()
```

این بلوک نقطه شروع برنامه است. یک پنجره **tkinter** ساخته می‌شود، آن را به **GUI** می‌دهد و با **mainloop()** برنامه را در حالت اجرا نگه می‌دارد تا کاربر پنجره را ببندد.